Training Guide

FMI Skid Demo — build, run, train, and produce a deployable supervisory network

A complete, copy-paste walkthrough of the training half of the pipeline: install the toolchain, build the platform, run the skid scenarios, train the four diagnostic heads, and produce the ready-to-deploy supervisory network the Deployment Guide consumes.

Document	Model Training Guide
Product	Jneopallium Industrial Loop Guardian (FMI Skid Demo)
Model	industrial-loop-guardian 1.0.0
Trainer	scripts/demo-industrial-fmi/ train_loop_guardian_model.py
Output	Deployable JNeopallium layer/neuron config
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1 What you will achieve

By the end of this guide you will have: a built Jneopallium platform; deterministic FMI skid runs across nine scenarios; a freshly trained **Industrial Loop Guardian** maintenance-and-energy model; and — the key output — a **complete, deployable JNeopallium network** (five layer files, real runtime classes, embedded trained heads, a fixed safety gate, and a ready-made production context) that the companion **Deployment Guide** loads directly.

> **Safety ceiling for the whole pipeline**
>
> Everything the trained model does is supervisory and ADVISORY. PLC/PID/SIS controls remain deterministic and authoritative; the trainer encodes this — the response layer is ADVISORY, hard safety gates are fixed configuration, and OPC UA actuator writes stay separate.

2 Where training ends and deployment begins

Phase	What happens	Document
Training (this guide)	Build, run scenarios, train on data, **emit the deployable network**	Training Guide
Deployment	Load the emitted network, configure event sources, run the advisory worker	Deployment Guide

The hand-off is a single directory of generated artifacts (Step 7). Training **produces** it; deployment **consumes** it. The trainer writes JNeopallium-ready layer and neuron configuration *and* a production context file, so the same files that record your training run are the files the worker boots from.

3 Prerequisites and system requirements

Toolchain

Tool	Version	Why
Java JDK	17 or newer (LTS)	Build and run the worker / controller
Apache Maven	3.9 or newer	Multi-module build of master + worker
Python	3.10+ (3.13 tested)	Runs the trainer, the demo runner, and report generator
Git	any recent	Clone the repository
Docker + Mosquitto (optional)	any recent	Only for the full OPC UA + MQTT protocol path

The default one-command runner produces deterministic evidence **without** Docker, a broker, or an OPC UA server. The full protocol path (gateway + `IndustrialFmiDemoMain`) additionally needs Maven, Mosquitto/Docker, the Python gateway dependencies, and a built FMU.

Verify the toolchain

```
java -version       # expect 17 or newer
mvn -version        # expect 3.9 or newer
python --version    # expect 3.10 or newer
```

4 Step 1 — Get the source and build

```
git clone https://github.com/rakovpublic/jneopallium.git
cd jneopallium
mvn clean install
```

The build installs `com.rakovpublic.jneopallium:worker:1.0`. The worker already contains the real industrial neuron, processor, and signal classes the trained network references (`com.rakovpublic.jneuropallium.worker.net.neuron.impl.industrial.*` and friends), so no extra hand-written classes are required to train.

5 Step 2 — Run the FMI skid scenarios

The deterministic runner advances the simulated skid and writes evidence for each scenario. Run all nine, or one at a time:

Linux / macOS

```
scripts/demo-industrial-fmi/run_demo.sh all
scripts/demo-industrial-fmi/run_demo.sh pump-wear
scripts/demo-industrial-fmi/run_demo.sh high-temperature-interlock
```

Windows (PowerShell)

```
powershell -ExecutionPolicy Bypass -File scripts/demo-industrial-fmi/run_demo.ps1 all
```

The nine scenarios are `normal`, `load-disturbance`, `oscillation`, `pump-wear`, `temperature-sensor-drift`, `mqtt-outage`, `opcua-outage`, `high-temperature-interlock`, and `operator-override`. Each exercises a different part of the supervisory and safety behaviour.

6 Step 3 — Understand the generated outputs

Each run writes a per-scenario folder under `target/`:

```
target/jneopallium-industrial-fmi/<scenario>/
  manifest.json            process_trace.csv        controller_results.jsonl
  advisory_findings.jsonl  model_advisory_findings.jsonl
  heuristic_advisory_findings.jsonl
  opcua_audit.jsonl        mqtt_audit.jsonl         alarms.jsonl
  interventions.jsonl      metrics.json             comparison.json
```

File	What it holds
`metrics.json`	Scenario KPIs: IAE, overshoot, settling, energy, interlock

	latency, availability
`comparison.json`	Baseline-versus-Jneopallium comparison
`model_advisory_findings.jsonl`	Advisories from the trained model
`advisory_findings.jsonl`	Advisories with owning neuron, economic basis, safety envelope, boundary
`opcua_audit.jsonl` / `mqtt_audit.jsonl`	Every command/telemetry event, for the protocol boundary

7 Step 4 — Train the Loop Guardian model

The trainer needs no third-party Python packages. Train and export the maintenance-and-energy model with the production-scale reference command:

```
python scripts/demo-industrial-fmi/train_loop_guardian_model.py \
  --reference-multiplier 1000 \
  --target-corpus-bytes 100gb \
  --max-corpus-bytes 100gb
```

It writes the full artifact set to `worker/src/main/resources/model/industrial-loop-guardian/` (see Step 7). The trainer fits four diagnostic heads over 39 engineered features, evaluates them on a leakage-safe split, and prints a summary including per-finding precision/recall/F1.

> **What the flags mean**
>
> `--reference-multiplier` deterministically expands the bundled skid corpus. `--target-corpus-bytes` records the logical production-scale target (100 GiB) reached by replication. `--max-corpus-bytes` is a hard guardrail that aborts rather than inflate the repository — the fitted sample stays a few tens of MiB so the run completes on a workstation.

8 Step 5 — Fast Python model verification

For a quick, build-free check of the model logic, run the Python unit tests:

```
python -m unittest discover -s scripts/demo-industrial-fmi/tests
```

These exercise the trainer and the loop-guardian model end to end without Maven, a broker, or a built FMU — ideal for a fast feedback loop while iterating on features or the corpus.

9 Step 6 — Generate training and run reports

After training and replay, render consolidated reports:

```
python scripts/demo-industrial-fmi/generate_run_reports.py
```

This collates the per-scenario metrics and the trained-model evidence into reproducible report artifacts you can attach to a pilot assessment.

10 Step 7 — The generated artifacts: a deployable network

This is the pipeline's hand-off. The training run writes a directory of JNeopallium-style configuration that is ready to deploy as-is — the actual layer-and-neuron network the worker boots from, plus a ready-made production context:

Artifact	What it is
`trained-industrial-loop-guardian-model.json`	The compact trained model: features, four heads, weights, metrics
`model-descriptor.json`	Whole-network descriptor: 5 layers, 17 neurons, frequency map, runtime classes
`layer-0.json`	Plant + supervisory-context input boundary
`layer-1-fast-telemetry.json`	Fast telemetry validation and loop state — 7 neurons
`layer-2-maintenance-energy.json`	The four trained diagnostic finding heads
`layer-3-advisory-planning.json`	Economic advisory planning + the fixed safety gate — 4 neurons
`result-layer.json`	Industrial advisory JSONL output
`trained-model-update.json`	Latest trained snapshot: status, checksum, weights, training summary
`quantitative-summary.json`	Scale, metrics, and top positive / negative feature weights per head
`source-mapping.json`	Which typed signals each data source maps onto
`production-context.json`	A ready-to-run advisory-worker context for deployment

The network the trainer builds

`model-descriptor.json` records a five-layer network of **17 real neurons**, with **156 trainable weights** and **4 biases**, every neuron referencing a concrete class from the worker's industrial package:

Layer	Size	Role
0 — Input	0	Plant / OPC UA / MQTT / FMI replay / Kafka boundary
1 — Fast telemetry	7	Validation, interlocks, override, fast loop state
2 — Diagnostic heads	4	Pump wear, oscillation/stiction, sensor drift, energy
3 — Advisory planning	4	Maintenance scheduling, bounded trim, economic basis, safety gate
4 — Result	2	Maintenance and energy advisories as JSONL

Each diagnostic head embeds its trained weights, the feature gate it is allowed to use (the oscillation head, for example, is gated to 17 of the 39 features), and its logical role and owned reasoning. The descriptor also carries a `signalFrequencyMap` giving each signal its loop cadence — fast tags every loop,

degradation/energy/setpoint every 10, maintenance windows every 60 — the multi-timescale design made concrete.

> **Why this matters**
>
> Because training emits real, inspectable, deployable configuration, there is no translation step between "the model we trained" and "the network we run." The Deployment Guide simply points the worker at this directory and loads `production-context.json`.

11 Step 8 — Verify the run against the gates

1. All expected source families are present in `source-mapping.json` (FMI replay, OPC UA, MQTT, Kafka shadow, CMMS, energy meter).
2. Per-finding F1 is at or near 1.0, and the energy head's false-positive rate is within budget (reference run: macro-F1 ≈ 0.9995, macro false-positive rate ≈ 0.0005).
3. The five layer files, `trained-model-update.json`, and `production-context.json` were written and reference the expected runtime classes.
4. Every scenario produced deterministic traces and a baseline comparison.
5. Advisory JSON includes the owning neuron, recommendation code, economic basis, safety-envelope result, and the PLC/PID/SIS-versus-Jneopallium boundary.

> **Reproducibility check**
>
> The reference run is deterministic. Re-running the same command must produce the same training checksum (`trained-model-update.json`) and the same per-finding metrics. A drift in either signals that an input or the toolchain changed.

12 Troubleshooting

Symptom	Likely cause and fix
`java -version` shows < 17	Install a JDK 17+ and set `JAVA_HOME`
Trainer cannot find scenarios	Run from the repo root; scenarios live in `scripts/demo-industrial-fmi/config/scenarios/`
"estimated canonical corpus exceeds --max-corpus-bytes"	Lower `--reference-multiplier`/`--target-corpus-bytes` or raise the guardrail deliberately
FMU / gateway errors	The default runner needs none of these; only the full protocol path requires the FMU + broker
Layer files not written	Confirm `--output-dir` is writable; the trainer writes all artifacts there
Unit tests fail after a change	A feature or corpus edit changed behaviour; review the diff and expected metrics

13 Appendix — training cheat-sheet

Build

```
git clone https://github.com/rakovpublic/jneopallium.git && cd jneopallium && mvn clean
install
```

Run scenarios

```
scripts/demo-industrial-fmi/run_demo.sh all            # Linux/macOS
powershell -File scripts/demo-industrial-fmi/run_demo.ps1 all   # Windows
```

Train, verify, report

```
python scripts/demo-industrial-fmi/train_loop_guardian_model.py \
  --reference-multiplier 1000 --target-corpus-bytes 100gb --max-corpus-bytes 100gb

python -m unittest discover -s scripts/demo-industrial-fmi/tests
python scripts/demo-industrial-fmi/generate_run_reports.py
```

Jneopallium Industrial Loop Guardian · Training Guide. The trainer emits a deployable JNeopallium network plus a production context; see the Deployment Guide to run it. Supervisory above PLC/PID/SIS. Safety mode: ADVISORY. License: BSD 3-Clause.